

Основы языка программирования Python¹

Регулярные выражения

Постановка задачи

Задан текст, прочитанный из файла:

```
1. foo = 10 ; bar = 0.01; baz = 40
236. bar = 0.01; baz= 30.1; foo =20.5
31. bar=0.01; foo = 30 ; baz =25.5
42. foo= 0.5; bar=10; baz=900
```

Требуется из каждой строки извлечь имена и значения встречающихся в тексте переменных (числа в начале строки игнорировать).

Регулярное выражение (**regular expressions**, сокр. **regex** или **regexr**) — это строка, предназначенная для описания формата текста, в котором ожидаются входные данные. Для этого используется особый мини-язык, включающий в себя определенный набор правил и синтаксических элементов.

Пример регулярного выражения

Регулярное выражение для описания формата электронной почты согласно стандарту RFC 5322:

```
( (?: [a-z0-9!#$%&'*/+=?^_`{|}~-] + (?: \. [a-z0-9!#$%&'*/+=?^_`{|}~-] +) * | \"(?: [\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f] | \\ [ \x01-\x09\x0b\x0c\x0e-\x7f] ) * \" ) @ (?: (?: [a-z0-9] (?: [a-z0-9-] * [a-z0-9] ) ? \. ) + [a-z0-9] (?: [a-z0-9-] * [a-z0-9] ) ? | \[ (?: (?: 25[0-5] | 2[0-4][0-9] | [01]?[0-9][0-9] ?) \. ) {3} (?: 25[0-5] | 2[0-4][0-9] | [01]?[0-9][0-9] ? | [a-z0-9-] * [a-z0-9] : (?: [\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f] | \\ [ \x01-\x09\x0b\x0c\x0e-\x7f] ) +) \] ) )
```

Подготовим данные для обработки

```
text_correct = """1. foo = 10 ; bar = 0.01; baz = 40
236. bar = 0.01; baz= 30.1; foo =20.5
31. bar=0.01; foo = 30 ; baz =25.5
42. foo= 0.5; bar=10; baz=900"""
```

```
text_invalid = """bar=0.01; baz=30; foo=10.2
5B. bar=0.01; baz=30; foo=10"""
```

```
lines_correct = text_correct.split("\n")
lines_invalid = text_invalid.split("\n")
```

Добавим проверку, что строка начинается с целого числа:

```
import re

...

regex = re.compile(r" *[0-9]+\..+")

for line in lines_correct:
    match = regex.match(line)
    assert match is not None

for line in lines_invalid:
    match = regex.match(line)
    assert match is None
```

Некоторые элементы языка регулярных выражений

7

Конструкция	Назначение
.	Любой символ
*	Предшествующее выражение должно встречаться 0 или более раз
+	Предшествующее выражение должно встречаться 1 или более раз
?	Предшествующее выражение должно встречаться 0 или 1 раз
{ m }	Предшествующее выражение должно встречаться ровно m раз
{ m , n }	Предшествующее выражение должно встречаться от m до n раз
	Оператор "или". Должно встретиться или выражение слева от символа " ", или выражение справа от него

Некоторые элементы языка регулярных выражений

8

Конструкция	Назначение
[. . .]	Перечисление символов, которые могут встретиться в этой позиции
[^ . . .]	Перечисление символов, которые не должны встречаться в этой позиции (не перечисленные символы могут располагаться в этой позиции)
\s	Любой пробельный символ
\S	Любой символ кроме пробельного
\d	Цифра
\D	Любой символ кроме цифры

Некоторые элементы языка регулярных выражений

9

Конструкция	Назначение
<code>\w</code>	Буквенный символ
<code>\W</code>	Любой символ кроме буквенного
<code>^</code>	Начало строки
<code>\$</code>	Конец строки

```
import re

...

regex = re.compile(r" *[0-9]+\..+")

for line in lines_correct:
    match = regex.match(line)
    assert match is not None

for line in lines_invalid:
    match = regex.match(line)
    assert match is None
```

Варианты использования конструкции перечисления

11

- `[0-9]`
- `[01234567]`
- `[a-z]`
- `[a-zA-Z]`
- `[0-9a-zA-Z]`
- `[0-9a-zA-Z_]*+`
- `[-0-9a-zA-Z_]*+`

Упростим регулярное выражение:

```
import re

...

regex = re.compile(r"\s*\d+\s*")

for line in lines_correct:
    match = regex.match(line)
    assert match is not None

for line in lines_invalid:
    match = regex.match(line)
    assert match is None
```

Параметры регулярных выражений

```
re.compile(pattern, flags=0)
```

Возможные значения параметра `flags` (параметры можно объединять с помощью оператора `"|"`):

- `re.NOFLAG`
- `re.A` или `re.ASCII`
- `re.DEBUG`
- `re.I` или `re.IGNORECASE`
- `re.M` или `re.MULTILINE`
- `re.S` или `re.DOTALL`
- `re.X` или `re.VERBOSE`

Добавим комментарии в регулярное выражение:

```
import re

...

regex = re.compile(r"""\s*\d+\.      # Пробел, число и точка
                  .+                # Пока любой символ""",
                  re.VERBOSE | re.IGNORECASE)

for line in lines_correct:
    match = regex.match(line)
    assert match is not None

for line in lines_invalid:
    match = regex.match(line)
    assert match is None
```

Регулярное выражение для извлечения остальных данных:

```
regex = re.compile(r"""  
    # Число в начале каждой строки  
    ^\s*\d+\.\s*  
    # Формат данных: XXX = NNM.MMM;  
    (\w+\s*=\s*\d+(\.\d+)?)\s*;\s*  
    (\w+\s*=\s*\d+(\.\d+)?)\s*;\s*  
    (\w+\s*=\s*\d+(\.\d+)?)\s*$""", re.VERBOSE)
```

Выведем значения групп:

```
import re

text = """1. foo = 10 ; bar = 0.01; baz = 40
236. bar = 0.01; baz= 30.1; foo =20.5
42. foo= 0.5; bar=10; baz=900"""

lines = text.split("\n")
regex = re.compile(r"""
    # Число в начале каждой строки
    ^\s*\d+\.\s*
    # Формат данных: XXX = NNM.MMM;
    (\w+\s*=\s*\d+(\.\d+)?)\s*;\s*
    (\w+\s*=\s*\d+(\.\d+)?)\s*;\s*
    (\w+\s*=\s*\d+(\.\d+)?)\s*$""", re.VERBOSE)

for line in lines:
    match = regex.match(line)
    assert match is not None
    print(match.group(0))
    print(match.groups())
    print(match.group(1), match.group(3), match.group(5),
          sep=", ", end="\n\n")
```



```
1. foo = 10 ; bar = 0.01; baz = 40  
( 'foo = 10', None, 'bar = 0.01', '.01', 'baz = 40', None)  
foo = 10, bar = 0.01, baz = 40
```

```
236. bar = 0.01; baz= 30.1; foo =20.5  
( 'bar = 0.01', '.01', 'baz= 30.1', '.1', 'foo =20.5', '.5')  
bar = 0.01, baz= 30.1, foo =20.5
```

```
42. foo= 0.5; bar=10; baz=900  
( 'foo= 0.5', '.5', 'bar=10', None, 'baz=900', None)  
foo= 0.5, bar=10, baz=900
```

Используем не захватывающие группы:

```
import re
```

```
text = """1. foo = 10 ; bar = 0.01; baz = 40
236. bar = 0.01; baz= 30.1; foo =20.5
42. foo= 0.5; bar=10; baz=900"""
```

```
lines = text.split("\n")
```

```
regex = re.compile(r"""
```

```
# Число в начале каждой строки
```

```
^\s*\d+\.\s*
```

```
# Формат данных: XXX = NNM.MMM;
```

```
(\w+\s*=\s*\d+(?:\.\d+)?)\s*;\s*
```

```
(\w+\s*=\s*\d+(?:\.\d+)?)\s*;\s*
```

```
(\w+\s*=\s*\d+(?:\.\d+)?)\s*$""", re.VERBOSE)
```

```
for line in lines:
```

```
    match = regex.match(line)
```

```
    assert match is not None
```

```
    print(match.group(0))
```

```
    print(match.groups())
```

```
    print(match.group(1), match.group(2), match.group(3),
          sep=", ", end="\n\n")
```

Результат выполнения:

```
1. foo = 10 ; bar = 0.01; baz = 40  
( 'foo = 10', 'bar = 0.01', 'baz = 40' )  
foo = 10, bar = 0.01, baz = 40
```

```
236. bar = 0.01; baz= 30.1; foo =20.5  
( 'bar = 0.01', 'baz= 30.1', 'foo =20.5' )  
bar = 0.01, baz= 30.1, foo =20.5
```

```
42. foo= 0.5; bar=10; baz=900  
( 'foo= 0.5', 'bar=10', 'baz=900' )  
foo= 0.5, bar=10, baz=900
```

Выделим имена переменных и их значения:

```
import re

text = """1. foo = 10 ; bar = 0.01; baz = 40
236. bar = 0.01; baz= 30.1; foo =20.5
42. foo= 0.5; bar=10; baz=900"""

lines = text.split("\n")
regex = re.compile(r"""
    # Число в начале каждой строки
    ^\s*\d+\.\s*
    # Формат данных: XXX = NNM.MMM;
    (?: (\w+)\s*=\s*(\d+(?:\.\d+)?))\s*;\s*
    (?: (\w+)\s*=\s*(\d+(?:\.\d+)?))\s*;\s*
    (?: (\w+)\s*=\s*(\d+(?:\.\d+)?))\s*$""",
    re.VERBOSE)

for line in lines:
    match = regex.match(line)
    assert match is not None
    print(match.group(0))
    print(match.groups(), "\n")
```

```
1. foo = 10 ; bar = 0.01; baz = 40  
( 'foo', '10', 'bar', '0.01', 'baz', '40' )
```

```
236. bar = 0.01; baz= 30.1; foo =20.5  
( 'bar', '0.01', 'baz', '30.1', 'foo', '20.5' )
```

```
42. foo= 0.5; bar=10; baz=900  
( 'foo', '0.5', 'bar', '10', 'baz', '900' )
```

Будем использовать именованные группы:

```
...
regex = re.compile(r"""
    # Число в начале каждой строки
    ^\s*\d+\.\s*
    # Формат данных: XXX = NNM.MMM;
    (?: (?P<name1>\w+)\s*=\s*(?P<val1>\d+(?:\.\d+)?) )\s*;\s*
    (?: (?P<name2>\w+)\s*=\s*(?P<val2>\d+(?:\.\d+)?) )\s*;\s*
    (?: (?P<name3>\w+)\s*=\s*(?P<val3>\d+(?:\.\d+)?) )\s*$""",
    re.VERBOSE)

for line in lines:
    match = regex.match(line)
    assert match is not None
    print(line)
    print(match.groupdict())
    print(f"{match.group('name1')} = {match.group('val1')}")
    print(f"{match.group('name2')} = {match.group('val2')}")
    print(f"{match.group('name3')} = {match.group('val3')}")
    print()
```

```
1. foo = 10 ;   bar  = 0.01; baz = 40
{'name1': 'foo', 'val1': '10', 'name2': 'bar', 'val2': '0.01', 'name3': 'baz', 'val3': '40'}
foo = 10
bar = 0.01
baz = 40
```

```
236.   bar = 0.01;   baz= 30.1; foo =20.5
{'name1': 'bar', 'val1': '0.01', 'name2': 'baz', 'val2': '30.1', 'name3': 'foo', 'val3': '20.5'}
bar = 0.01
baz = 30.1
foo = 20.5
```

```
42. foo= 0.5;   bar=10;   baz=900
{'name1': 'foo', 'val1': '0.5', 'name2': 'bar', 'val2': '10', 'name3': 'baz', 'val3': '900'}
foo = 0.5
bar = 10
baz = 900
```

Методы класса `Pattern` для поиска и замены

24

- `search()` — поиск первого совпадения. Можно указывать интервал поиска в строке. Возвращает объект `Match`, если совпадение найдено, или `None` в противном случае.
- `findall()` — возвращает список найденных подстрок или кортежей из подстрок, если регулярное выражение содержит группы.
- `finditer()` — возвращает итерируемый объект, который возвращает объекты класса `Match`, описывающие найденные подстроки.
- `sub()` — возвращает строку после замены найденной подстроки на заданную строку.

Поиск с использованием регулярного выражения:

```
import re
```

```
text = """1. foo = 10 ; bar = 0.01; baz = 40
5XX. bar=0.01; baz=30; foo=10
236. bar = 0.01; baz= 30.1; foo =20.5
bar=0.01; baz=30; foo=10.2
42. foo= 0.5; bar=10; baz=900"""
```

```
regex = re.compile(r"""
    # Число в начале каждой строки
    ^\s*\d+\.\s*
    # Формат данных: XXX = NNM.MMM;
    (?: (?:P<name1>\w+)\s*=\s*(?:P<val1>\d+(?:\.\d+)?) )\s*;\s*
    (?: (?:P<name2>\w+)\s*=\s*(?:P<val2>\d+(?:\.\d+)?) )\s*;\s*
    (?: (?:P<name3>\w+)\s*=\s*(?:P<val3>\d+(?:\.\d+)?) )\s*$""",
    re.VERBOSE | re.MULTILINE)
```

```
for match in regex.finditer(text):
    print(match.group(0))
    print(match.groupdict())
    print(f"{match.group('name1')} = {match.group('val1')}")
    print(f"{match.group('name2')} = {match.group('val2')}")
    print(f"{match.group('name3')} = {match.group('val3')}")
    print()
```

Результат выполнения:

```
1. foo = 10 ;   bar  = 0.01; baz = 40
{'name1': 'foo', 'val1': '10', 'name2': 'bar', 'val2': '0.01', 'name3': 'baz', 'val3': '40'}
foo = 10
bar = 0.01
baz = 40
```

```
236.   bar = 0.01;   baz= 30.1; foo =20.5
{'name1': 'bar', 'val1': '0.01', 'name2': 'baz', 'val2': '30.1', 'name3': 'foo', 'val3': '20.5'}
bar = 0.01
baz = 30.1
foo = 20.5
```

```
42. foo= 0.5;   bar=10;   baz=900
{'name1': 'foo', 'val1': '0.5', 'name2': 'bar', 'val2': '10', 'name3': 'baz', 'val3': '900'}
foo = 0.5
bar = 10
baz = 900
```

Будем искать только выражения вида "XXX = NNN.MMM":

27

```
import re
```

```
text = """1. foo = 10 ; bar = 0.01; baz = 40
236. bar = 0.01; baz= 30.1; foo =20.5
42. foo= 0.5; bar=10; baz=900"""
```

```
regex = re.compile(r"""(?P<name>\w+)\s*=\s*(?P<val>\d+(?:\.\d+)?)""",
                    re.MULTILINE)
```

```
for match in regex.finditer(text):
    print(match.groupdict())
```

Результат выполнения:

```
{ 'name': 'foo', 'val': '10' }  
{ 'name': 'bar', 'val': '0.01' }  
{ 'name': 'baz', 'val': '40' }  
{ 'name': 'bar', 'val': '0.01' }  
{ 'name': 'baz', 'val': '30.1' }  
{ 'name': 'foo', 'val': '20.5' }  
{ 'name': 'foo', 'val': '0.5' }  
{ 'name': 'bar', 'val': '10' }  
{ 'name': 'baz', 'val': '900' }
```

Инструкции подстановки при использовании метода `sub()`

- `\g<groupname>` — для подстановки подстроки, попавшей в именованную группу с именем `groupname`.
- `\1... \9` — для подстановки подстрок, попавших в не именованные группы.
- `\g<NN>` — для подстановки подстрок, попавших в не именованные группы с номером `NN`.
- `\g<0>` — для постановки полной найденной подстроки.

Удалим пробелы до и после знаков "=":

```
import re
```

```
text = """1. foo = 10 ; bar = 0.01; baz = 40  
236. bar = 0.01; baz= 30.1; foo =20.5  
42. foo= 0.5; bar=10; baz=900"""
```

```
regex = re.compile(r"""(?P<name>\w+)\s*=\s*(?P<val>\d+(?:\.\d+)?)"",  
                  re.MULTILINE)
```

```
result = regex.sub(r"\g<name>=\g<val>", text)  
print(result)
```

```
1. foo=10 ; bar=0.01; baz=40
236. bar=0.01; baz=30.1; foo=20.5
42. foo=0.5; bar=10; baz=900
```

Удалим пробелы до и после знаков "=" без использования именованных групп:

```
import re
```

```
text = """1. foo = 10 ; bar = 0.01; baz = 40
236. bar = 0.01; baz= 30.1; foo =20.5
42. foo= 0.5; bar=10; baz=900"""
```

```
regex = re.compile(r"""(\w+)\s*=\s*(\d+(?:\.\d+)?)""",
                  re.MULTILINE)
```

```
result = regex.sub(r"\1=\2", text)
print(result)
```


Использование функций модуля re вместо методов класса

Pattern:

```
import re
```

```
text = """1. foo = 10 ; bar = 0.01; baz = 40
236. bar = 0.01; baz= 30.1; foo =20.5
42. foo= 0.5; bar=10; baz=900"""
```

```
regex = r"""(?P<name>\w+)\s*=\s*(?P<val>\d+(?:\.\d+)?)"""
result = re.sub(regex, r"\g<name>=\g<val>", text, flags=re.MULTILINE)
print(result)
```